



DSA Notes

Space Complexity — Beginner's Cheatsheet



1. What is Space Complexity?

Space Complexity = how much memory an algorithm uses as the input size (n) grows.

It measures all the memory your algorithm touches:

- Variables declared (let x = 0)
- Arrays or data structures created
- Objects / maps / sets
- Function call stack (especially in recursion)
- Temporary / helper memory



Key idea: Just like Time Complexity measures how STEPS grow, Space Complexity measures how MEMORY grows.



2. The Formula

Space Complexity = Input Space + Auxiliary Space

Auxiliary Space = extra temporary memory used beyond the input itself

Breaking it down:

- **Input Space:** memory used to store the input itself (the array, string, or number passed in)
- **Auxiliary Space:** extra memory your algorithm creates while running (temp arrays, recursion stack, helper vars)



In most interviews, when people say 'Space Complexity', they usually mean Auxiliary Space — the extra memory beyond the input.

3. What is Auxiliary Space?

Auxiliary Space is the extra temporary memory used by the algorithm — not counting the input.

Concept	Meaning
Space Complexity	TOTAL memory used (input + extra)
Auxiliary Space	Only the EXTRA memory beyond the input

Real-world example:

Algorithm	Auxiliary Space
Merge Sort — creates a temp array to merge	$O(n)$ extra
Bubble Sort — swaps in-place, no temp array	$O(1)$ extra

✓ Bubble Sort wins on space because it needs zero extra memory. Merge Sort is faster but costs $O(n)$ auxiliary space.

4. Constant Space — $O(1)$

The Code:

```
function sum(n) {  
  return n * (n + 1) / 2;  // math formula  
}
```

Analysis:

- No array, no extra structure created
- Just uses a couple of CPU registers for the arithmetic
- Memory used is **fixed** — does NOT grow with n

👉 No matter if $n = 10$ or $n = 1,000,000$ — the same amount of memory is used.

✓ Space Complexity:
 $O(1)$

5. Loop with Variables — Still $O(1)$

The Code:

```
function sum(n) {  
  let total = 0;           // one variable  
  for (let i = 1; i <= n; i++) {  
    total += i;  
  }  
  return total;  
}
```

Analysis:

- The loop runs n times — but that affects **TIME, not SPACE**
- Only **two variables** exist at any moment: `total` and `i`
- Variable count is FIXED regardless of n

👉 Loops affect **TIME**. They do **NOT** create new memory — the same two variables are reused every iteration.

✓ Space Complexity:
 $O(1)$

6. Array Space — $O(n)$

The Code:

```
function createArray(n) {  
  let arr = [];
```

```

for (let i = 0; i < n; i++) {
  arr.push(i);      // adds one slot per step
}
return arr;
}

```

Analysis:

- Every `arr.push(i)` adds one more element to memory
- If $n = 100 \rightarrow$ array uses 100 slots of memory
- If $n = 1000 \rightarrow$ array uses 1000 slots of memory
- Memory grows **linearly** with n

👉 An array that grows as n grows = $O(n)$ space. Each `push()` reserves new memory.

✓ Space Complexity:
 $O(n)$

7. 2D Matrix Space — $O(n^2)$

The Code:

```

let matrix = [];
for (let i = 0; i < n; i++) {
  matrix[i] = [];
  for (let j = 0; j < n; j++) {
    matrix[i][j] = 0;    // n × n cells total
  }
}

```

Analysis:

- n rows, each row has n columns $\rightarrow$ total = $n \times n$ cells
- Every single cell = one unit of memory
- If $n = 10 \rightarrow 100$ cells. If $n = 100 \rightarrow 10,000$ cells
- Memory grows **quadratically** — gets expensive fast

👉 A 2D grid of size $n \times n$ stores n^2 values. Double $n \rightarrow$ memory quadruples.

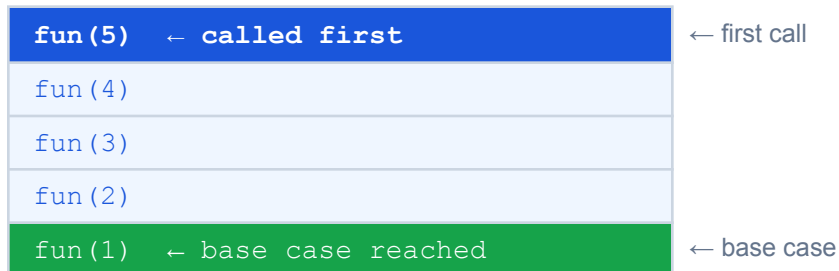
✓ Space Complexity:
 $O(n^2)$

8. Recursion Stack Space — $O(n)$

The Code:

```
function fun(n) {  
  if (n <= 0) return; // base case  
  fun(n - 1);         // calls itself with n-1  
}
```

Each call waits for the next — they stack up in memory like plates:



Analysis:

- fun(n) calls fun(n-1), which calls fun(n-2) ... all the way to fun(0)
- Each unfinished call **stays in memory** waiting for the one below it to finish
- At peak: n frames are alive simultaneously on the call stack

👉 The call stack acts like a pile. You push a call on top, and only pop it off when it returns. Recursion n-1 deep = n frames = $O(n)$ stack space.

✓ Space Complexity:
 $O(n)$

9. Logarithmic Recursive Space — $O(\log n)$

The Code:

```
function fun(n) {  
  if (n <= 1) return; // base case  
  fun(n / 2); // calls itself with HALF the input  
}
```

Stack for $n = 64$ — input halves every call, so only $\log_2(64) = 6$ frames:

fun(64) ← called first	← first call
fun(32)	
fun(16)	
fun(8)	
fun(4)	
fun(2)	
fun(1) ← base case	← base case

Analysis:

- Each call halves $n \rightarrow$ sequence: $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$
- How many steps until we reach 1? $\rightarrow \log_2(n)$ steps
- Far fewer stack frames than the $n-1$ version — much more memory efficient

👉 Halving input = logarithmic depth. For $n = 1,000,000$: $O(n)$ recursion uses 1,000,000 frames. $O(\log n)$ uses only ~20 frames!

✓ Space Complexity:
 $O(\log n)$

10. Time Complexity vs Space Complexity

Time Complexity	Space Complexity
Measures how STEPS grow with n	Measures how MEMORY grows with n
Loops affect it most	Arrays & recursion affect it most

Single loop = $O(n)$ time	Single loop = $O(1)$ space
Nested loop = $O(n^2)$ time	Nested array/matrix = $O(n^2)$ space
Recursion $(n-1) = O(n)$ time	Recursion $(n-1) = O(n)$ space

 **Most important insight: A LOOP adds to time complexity but NOT to space ($O(1)$ space). An ARRAY or RECURSION adds to BOTH.**

11. Common Space Complexities — Quick Reference

Pattern	Space
Variables only	$O(1)$
Array of size n	$O(n)$
Matrix $n \times n$	$O(n^2)$
Recursion ($n-1$ each call)	$O(n)$ stack
Recursion ($n/2$ each call)	$O(\log n)$ stack

12. Final Takeaway

Final Takeaway

- **Space Complexity** = how much memory grows with input n .
- **Auxiliary Space** = only the EXTRA memory beyond the input itself.
- **Variables / fixed counters** = $O(1)$ — they never grow.
- **Arrays growing with n** = $O(n)$ — each element adds memory.
- **$n \times n$ matrices** = $O(n^2)$ — every row and column adds up.
- **Recursion ($n-1$ each call)** = $O(n)$ stack — n frames pile up.
- **Recursion ($n/2$ each call)** = $O(\log n)$ stack — frames halve quickly.
- **Loops alone** = $O(1)$ space — they don't add memory, only time!